

PROMPT IA 10

Customer Storefront complet

Next.js + TypeScript + Tailwind CSS + composants maison/headless

Design Pack NOMA comme source visuelle de reference

Objectif : transformer les maquettes valides du Design Pack en Storefront e-commerce reel, mobile-first, accessible, rapide, maintenable et branche sur les APIs backend existantes.

Token	Valeur de reference	Usage
Ivoire chaud	#F7F3EC	fonds chauds / sections
Blanc	#FFFFFF	surfaces
Charbon	#17202A	texte principal
Bleu electrique	#2764FF	CTA / focus / accent principal
Corail	#FF6B55	promo / erreur selective
Gris doux	#8A9099	texte secondaire / neutral

Core Web Vitals prioritaires - JavaScript maitrise et justifie - backend authoritative

1. Role de l'IA

Tu es Principal Frontend Engineer et UI Engineer senior. Tu dois implementer le Customer Storefront complet dans le monorepo existant en respectant strictement le Design Pack NOMA, les contrats backend et les contraintes de performance. Tu dois produire du code reel, compilable, teste, accessible et maintenable - pas une maquette statique.

Le Design Pack fourni dans design-pack-storefront-noma(1).zip est la source visuelle de reference. Les PNG sont des references de direction et de composition ; corrige les incoherences raster/texte et transpose-les en composants, tokens et comportements accessibles plutot que de les copier pixel par pixel.

2. Fichiers du Design Pack a utiliser

- design-pack-storefront/design-pack-storefront-prompt.pdf - brief source et contraintes.
- maquettes/01-direction-artistique.png - palette, typographie, composants, spacing, focus, photographie.
- maquettes/02-home-mobile.png et 03-home-desktop.png - Home mobile-first et adaptation desktop.
- maquettes/04-catalogue-mobile.png et 05-catalogue-desktop.png - catalogue, filtres et grille.
- maquettes/06-product-mobile.png et 07-product-desktop.png - fiche produit, galerie, variantes et achat.

Ne remplace pas l'identite NOMA par un template generique. La marque reste remplaçable via tokens/contenu, mais la realisation doit correspondre au Design Pack fourni.

3. Stack obligatoire

Next.js App Router
TypeScript strict
React Server Components / SSR par default
Tailwind CSS
CSS variables + design tokens
Composants maison / headless
Flexbox + CSS Grid
fetch / Server Actions lorsque justifie
SSE/WebSocket seulement si besoin reel
Tests unitaires + integration + E2E
OpenTelemetry / RUM / Web Vitals

Ne pas introduire Bootstrap, Materialize, MUI, styled-components ou une grosse bibliotheque UI/runtime CSS-in-JS. Toute nouvelle dependance frontend lourde doit avoir un benefice mesurable.

4. Principes UI obligatoires

4.1 Mobile-first

La base fonctionnelle doit fonctionner a 320 px, avec cible principale autour de 390 px. Le desktop est une adaptation enrichie, pas une autre application.

base CSS = mobile
sm/md = enrichissement progressif
lg/xl = desktop
max-width controlee
aucun overflow horizontal

4.2 Flexbox et Grid

Utiliser Flexbox pour layouts lineaires (header, actions, lignes panier, controles) et CSS Grid pour layouts 2D (catalogue, galerie, zones editoriales). Ne pas forcer Grid ou Flex lorsqu'un flux normal suffit.

4.3 Accessibilite

- WCAG 2.2 AA
- corps mobile ≥ 16 px
- cibles tactiles $\geq 44 \times 44$ px
- focus clavier visible
- navigation clavier complete
- labels et erreurs explicites

- information jamais uniquement par couleur
- prefers-reduced-motion
- sémantique HTML correcte et landmarks

5. Design system NOMA

Construire le design system a partir de 01-direction-artistique.png. Les valeurs ci-dessous sont les references initiales ; si une maquette revele une valeur plus coherente, conserver une seule source de verite dans les tokens.

```
--color-ivory: #F7F3EC
--color-white: #FFFFFF
--color-charcoal: #17202A
--color-primary: #2764FF
--color-coral: #FF6B55
--color-muted: #8A9099
```

spacing: 8,16,24,32,40,48,64 px
 radius principal: 12 px
 photo: lumiere naturelle douce, fonds neutres, focus produit

Typographie de reference : sans-serif geometrique/humaniste, lisible et chaleureuse. H1 environ 32/40, H2 24/32, H3 20/28, body 16/24, small 14/20. Limiter les graisses et variantes chargees.

6. Strategie performance - Core Web Vitals prioritaires

La regle n'est pas "zero JavaScript", mais "zero JavaScript inutile". Autoriser du JavaScript client quand il ameliore reellement UX ou performance perçue, et mesurer son cout.

Metrique	Cible	Regle
LCP	< 2,5 s	image hero/product priorisee, dimensions reservees
CLS	< 0,1	ratios stables, pas de contenu injecte sans espace reserve
INP	< 200 ms	interactions courtes, composants client isoles
JS	budget controle	code splitting, dynamic import, dependances justifiees

- RSC/SSR par default
- Client Components uniquement aux zones interactives
- images AVIF/WebP, srcset/sizes et ratio reserve
- font loading optimise et sous-ensemble minimal
- streaming/Suspense quand cela reduit le temps perçu
- prefetch cible, jamais massif
- lazy loading hors viewport
- RUM avec web-vitals en production

7. Architecture frontend et trust boundaries

Browser
 -> ATS cache/CDN
 -> Next.js Storefront
 -> RSC/SSR + BFF
 -> APIs REST via Kong
 -> Search read model / backend services

Frontend = affichage, collecte, prevalidation UX

Backend = authentication, autorisation, calcul, validation, decision, persistance, confirmation

Le navigateur ne confirme jamais prix final, stock final, paiement, commande, remboursement, retour ou permissions. Toute decision critique est validee et confirmee par le backend authoritative.

8. Pages a implementer

Zone	Pages / fonctions principales
Discovery	Home, navigation, categories, search, autocomplete, catalogue, filtres/facettes, tri.
Product	fiche produit, galerie, variantes, prix indicatif, disponibilite, avis synthese, ajout panier.
Cart	mini-cart, panier, quantites, suppression, prix indicatifs, messages disponibilite.
Checkout	adresse, livraison, recapitulatif, paiement, etats de traitement, confirmation backend.
Account	connexion, compte, commandes, detail commande, tracking, retours, avis.
States	loading, skeleton, empty, error, offline/degraded, unavailable, retry.

9. Home

Reproduire la direction des maquettes Home mobile/desktop : header clair, recherche immediatement accessible, proposition de valeur concise, editorial leger, categories, selection produits, reassurance, contenu de decouverte et footer compact.

- aucun carrousel obligatoire
- pas de hero video automatique
- contenu essentiel au-dessus de la ligne de flottaison
- navigation peu profonde
- une action principale claire
- desktop enrichi sans rompre la hierarchy mobile

10. Catalogue / PLP

Respecter les maquettes catalogue. Mobile : controls filtres/tri compacts et accessibles. Desktop : filtres lateraux et grille 4 colonnes lorsque la largeur le permet. La grille doit se degrader proprement selon le viewport.

mobile: 1-2 colonnes selon largeur reelle
 tablette: 2-3 colonnes
 desktop: jusqu'a 4 colonnes

ProductCard = image stable + nom + note + prix + badges + action secondaire
 aucun layout shift au chargement

Filtres et pagination doivent etre compatibles avec URLs partageables, back/forward browser et rendu serveur. Favoriser query params et pagination par curseur lorsque le backend le permet.

11. Product / PDP

Respecter les maquettes Product mobile/desktop : galerie claire, titre, rating, prix, variantes, disponibilite, CTA achat, details et informations de confiance. Sur desktop, panneau achat lisible ; sur mobile, ordre de contenu priorise pour une main.

- image principale priorisee pour LCP
- galerie sans bibliotheque carousel lourde
- selecteurs variante accessibles
- quantity control 44px+
- CTA visible sans masquer du contenu
- prix/stock affiches comme indicatifs puis revalides backend

12. Cart et interaction AJAX/fetch

Utiliser fetch/Server Actions de facon ciblee pour ajouter, modifier ou supprimer des articles sans rechargement complet. Optimistic UI uniquement lorsque le rollback est simple et que le backend reste authoritative.

```
add item -> UI pending -> backend Cart -> success => confirm UI
-> error => rollback + message explicite
```

Utiliser AbortController pour recherches/autocomplete et debounce raisonnable. Eviter les appels repetitifs non annules.

13. Checkout

Construire un checkout mobile-first, progressif et sans dark pattern. Les etapes doivent utiliser les services backend deja implementes : Cart, Pricing, Tax, Inventory, Fraud/Risk, Order, Payment/Hyperswitch, Shipping et Address/Pelias-libpostal.

```
Cart
-> address input / Pelias autocomplete
-> shipping option
-> backend recalcul Pricing + Tax
-> backend Inventory reservation
-> Fraud/Risk
-> Payment/Hyperswitch
-> Order confirmed
-> UI confirmation
```

Un message de succes du navigateur ou du PSP ne suffit jamais. La page de confirmation attend un etat confirme par le backend.

14. Adresse et Pelias/libpostal

Autocomplete adresse asynchrone avec debounce, annulation des requetes et navigation clavier. Si Pelias est indisponible, la saisie manuelle reste possible. Ne bloque jamais le checkout uniquement parce que l'autocomplete est down.

15. Authentification et compte

Integrer le flux IAM du Prompt 09 : Keycloak OIDC/OAuth2, PKCE, MFA/Passkeys lorsque applicable. Ne stocker aucun secret d'auth dans localStorage. Respecter la strategie sessions/tokens du backend.

Pages : login/register si UX locale necessaire, verification, MFA/passkey, compte, commandes, detail, tracking, returns, reviews. Les permissions backend restent authoritative.

16. Tracking et temps reel

Pour Tracking, privilegier une lecture SSR/RSC initiale puis SSE si un besoin de mise a jour quasi temps reel est valide. Eviter le polling agressif. WebSocket uniquement si un vrai flux bidirectionnel le justifie.

17. Cache et freshness

Donnee	Lecture affichee	Decision transactionnelle
Produit/catalogue	ATS/Next.js/Search/OpenSearch	Product backend si besoin
Prix	projection/cache possible	Pricing recalcule au checkout
Stock	indication stale toleree	Inventory verifie/reserve
Reviews	eventual consistency	non critique
Order/Payment	backend authoritative	backend authoritative

Definir explicitement les revalidate/cache-control selon la nature des donnees et les invalidations evenementielles disponibles.

18. Composants a factoriser

```
ui/
Button
IconButton
Input / Select / Checkbox
SearchBox
Price
Badge
Rating
```

```
ProductCard
ProductGrid
QuantityControl
Breadcrumb
FilterPanel / FilterDrawer
Pagination
Skeleton
EmptyState / ErrorState
Modal/Drawer headless
Toast accessible

commerce/
MiniCart
CartLine
ProductGallery
VariantSelector
AddressAutocomplete
CheckoutSummary
```

Ne factorise pas prematurement des composants qui n'ont aucun usage multiple. Eviter les mega-components et prop drilling excessif.

19. Etat, formulaires et validation

Preferer l'etat local au niveau le plus proche. Ne pas introduire un store global lourd par reflexe. Utiliser une librairie de formulaire uniquement si elle simplifie nettement le checkout/validation.

Validation frontend = UX. Validation backend = autorite. Afficher les erreurs backend de facon claire et associee aux champs.

20. SEO et metadata

- metadata Next.js par page
- canonical URLs
- Open Graph/Twitter cards
- JSON-LD Product/Breadcrumb/Organization lorsque valide
- sitemap et robots
- pagination/indexation controlee
- 404/410 coherents
- URLs lisibles et stables

Ne pas generer de donnees structurees mensongeres : prix, stock et notes doivent provenir de donnees backend valides.

21. Images et medias

Utiliser next/image ou pipeline equivalent avec tailles explicites, sizes/srcset et formats modernes. Conserver la direction photo NOMA : lumiere naturelle douce, fonds neutres, materiaux visibles, angles simples et focus produit.

- pas d'image gigantesque sur mobile
- preload uniquement image LCP
- lazy pour le hors viewport
- placeholder/skeleton stable
- alt text fonctionnel
- pas de texte critique rasterise dans les images

22. Tests obligatoires

Categorie	Tests minimum
Unit/component	formatters, composants interactifs, validators UX.

Categorie	Tests minimum
Integration	BFF/API clients, cache/revalidation, auth boundaries.
E2E Playwright	Home -> Search -> Product -> Cart -> Checkout -> Confirmation ; account/order/return.
Accessibility	axe ou equivalent + clavier manuel automatise sur parcours critiques.
Visual	screenshots mobile/desktop compares au Design Pack avec tolerance raisonnee.
Performance	Lighthouse CI + Web Vitals + bundle budget.
Failure	Search down, Pelias down, notification retardee, payment pending/error, backend validation error.

23. Observabilite frontend

Instrumenter erreurs JS, navigation, fetch critiques et Web Vitals. Propager correlation/request IDs lorsque le backend les expose. Ne jamais envoyer PII/secrets dans la telemetrie frontend.

```
web_vital_lcp
web_vital_cls
web_vital_inp
route_render_duration
frontend_api_error_total
checkout_step_error_total
search_latency_client
js_error_total
```

24. Security frontend

- CSP compatible avec les besoins reels
- aucun secret dans bundle public
- cookies/session selon strategie IAM
- CSRF selon mode d'auth
- XSS : echappement et pas de dangerouslySetInnerHTML non controle
- dependances scannees
- backend revalide toutes les permissions
- aucune confiance aux prix/montants provenant du DOM/client

25. Structure recommandee

```
apps/storefront/
app/
(public)/
account/
checkout/
components/
ui/
commerce/
layout/
lib/
api/
auth/
observability/
seo/
styles/
tokens.css
tests/
public/
```

Eviter les dossiers fourre-tout et les helpers dupliques.

26. Ordre d'implementation

1. Importer/extraire les design tokens NOMA
2. Layout global + Header/Footer + typography
3. Home mobile puis desktop
4. Catalogue mobile puis desktop
5. Product mobile puis desktop
6. Brancher Search/Product/Pricing/Inventory
7. Cart + mini-cart
8. Checkout + address + shipping + payment
9. Auth/account/orders/tracking/returns/reviews
10. Error/loading/empty/degraded states
11. Accessibility pass
12. Visual regression vs Design Pack
13. Performance budgets + Lighthouse/RUM
14. CI/CD + ATS/cache headers
15. Documentation + runbooks frontend

27. Definition of Done

- Home, Catalogue et Product reproduisent fidelement la direction NOMA sur mobile et desktop.
- Le storefront fonctionne de 320 px aux grands desktops sans overflow ni rupture.
- Le parcours Search -> Product -> Cart -> Checkout -> Order confirmed passe en E2E.
- Prix, stock, paiement et commande sont confirmes par le backend, jamais par le client.
- Les interactions essentielles sont accessibles clavier et WCAG 2.2 AA.
- LCP/CLS/INP respectent les budgets sur les pages critiques dans les conditions de test definies.
- Le JavaScript client est mesure, justifie et fractionne ; aucun gros runtime UI inutile.
- Les images sont responsives et n'introduisent pas de layout shift.
- Les erreurs et modes degrades sont utilisables, pas seulement logges.
- Les tests unitaires, integration, E2E, accessibilite, visual regression et performance passent.
- Le code est factorise. type-safe. lint clean et deployable par la CI/CD existante.

Ne declare pas le Storefront termine parce que les pages "ressemblent" aux maquettes. Il est termine lorsque le Design Pack est transpose en design system coherent, que les parcours metier sont branches au backend, que l'accessibilite et les Core Web Vitals sont verifiees et que les tests E2E passent.

28. Regles de travail pour l'IA

Commence par analyser le Design Pack fourni avant de coder. Travaille par increments visibles. Apres chaque increment : typecheck, lint, tests, screenshot mobile/desktop, comparaison au Design Pack, correction, puis factorisation. Ne repete pas la meme logique dans plusieurs composants. Toute modification significative doit rester dans un diff coherent et maintenable.

Si un detail raster est ambigu, privilegie les tokens, le brief source, l'accessibilite et la performance. Ne copie jamais une erreur de texte, contraste ou alignement provenant d'une maquette conceptuelle.